
SOFTWARE DEVELOPER TRAINING

Curso 1: Introducción al Software y su Historia

Tipo: Fundamento teórico (modificación del curso existente)

Fase: 1 — Fundamentos teóricos

Modulos: 6

Objetivo general

Comprender qué es el software, cómo ha evolucionado a lo largo de la historia, los diferentes tipos que existen, los ecosistemas tecnológicos donde opera, los roles profesionales involucrados en su desarrollo, y dar el primer paso práctico escribiendo código. Al finalizar, el estudiante tendrá una visión panorámica completa del mundo del desarrollo de software.

Nota importante

El contenido presentado en este curso es introductorio y orientativo. Su propósito es proporcionar una base conceptual y práctica sobre cada tema, pero no pretende ser exhaustivo. Se espera que el estudiante complemente este material con su propia investigación, consulte fuentes adicionales (documentación oficial, artículos, tutoriales) y profundice en los temas según su ritmo y curiosidad. La capacidad de investigar y aprender de forma autónoma es una habilidad fundamental en el desarrollo de software.

Módulo 1: ¿Qué es el software? Hardware vs Software

Contenido

- **Definición de software:** conjunto de instrucciones, datos y programas que le indican al hardware qué hacer. Software como capa lógica sobre la capa física.
- **Definición de hardware:** componentes físicos de un sistema de cómputo.
 - **Procesamiento:** CPU (núcleos, hilos, velocidad de reloj), GPU (procesamiento paralelo, uso en gráficos e IA).
 - **Memoria:** RAM (volátil, velocidad), ROM (firmware).
 - **Almacenamiento:** HDD vs SSD, almacenamiento en la nube.
 - **Periféricos de entrada:** teclado, mouse, micrófono, cámara, scanner.
 - **Periféricos de salida:** monitor, impresora, parlantes.
 - **Periféricos de entrada/salida:** pantallas táctiles, dispositivos de red.
 - **Placa madre:** componente que conecta todo.
- **Relación simbiótica:** el hardware sin software es un objeto inerte; el software sin hardware no puede ejecutarse. Uno no funciona sin el otro.
- **Capas de interacción entre hardware y software:**
 - **Firmware:** software embebido en el hardware (BIOS/UEFI).
 - **Drivers:** software que permite al SO comunicarse con el hardware.
 - **Sistema operativo:** capa intermediaria entre el usuario/aplicaciones y el hardware.
 - **Aplicaciones:** software que el usuario final utiliza.
- **Ejemplos cotidianos:**
 - WhatsApp (software) ejecutándose en un smartphone (hardware).
 - Microsoft Word (software) ejecutándose en un laptop (hardware).
 - El sistema de navegación GPS de un auto (software embebido en hardware automotriz).

Actividad del modulo

1. Elaborar un **mapa conceptual** que represente la relación entre hardware y software, incluyendo: las capas de interacción (firmware, drivers, SO, aplicaciones), al menos 5 componentes de hardware con su función, y al menos 3 ejemplos cotidianos de la relación hardware-software.
2. Responder por escrito: "¿Qué pasaría si tuvieras hardware sin software? ¿Y software sin hardware?" (mínimo un párrafo por pregunta).

Módulo 2: Historia del software — de los pioneros a la era moderna

Contenido

- **Los pioneros (siglo XIX - 1940s):**
 - **Ada Lovelace (1843):** primera programadora de la historia. Su trabajo con la Máquina Analítica de Charles Babbage. El primer algoritmo diseñado para ser ejecutado por una máquina.
 - **Alan Turing (1936):** la Máquina de Turing, concepto de computación universal. Su papel en la Segunda Guerra Mundial descifrando Enigma.
 - **ENIAC (1945):** primer computador electrónico de propósito general. Programado mediante cables y switches.
- **La era de los lenguajes (1950s - 1960s):**
 - **Grace Hopper:** creadora del primer compilador (A-0). Pionera de COBOL.
 - **FORTRAN (1957):** primer lenguaje de alto nivel, orientado a cálculos científicos.
 - **COBOL (1959):** orientado a negocios y administración.
 - **LISP (1958):** orientado a inteligencia artificial.
 - Concepto de compilador e intérprete: traducir código humano a código máquina.
- **La revolución de los sistemas operativos (1970s - 1980s):**
 - **UNIX (1969):** sistema operativo que sentó las bases de Linux y macOS.
 - **Lenguaje C (1972):** creado por Dennis Ritchie, usado para reescribir UNIX. Base de muchos lenguajes modernos.
 - **MS-DOS (1981) y Windows (1985):** la computadora personal llega al consumidor.
 - **Apple Macintosh (1984):** interfaz gráfica de usuario (GUI) para las masas.
 - Software empresarial: inicio de SAP, Oracle, bases de datos relacionales.
- **La era de internet (1990s - 2000s):**
 - **World Wide Web (1991):** Tim Berners-Lee. HTML, HTTP, el navegador.
 - **Java (1995):** "Write once, run anywhere". Applets, software empresarial.
 - **JavaScript (1995):** interactividad en el navegador. De lenguaje "de juguete" a pilar de la web.
 - **PHP, MySQL:** el stack de la web dinámica (WordPress, Facebook en sus inicios).
 - **Open Source:** Linux, Apache, movimiento de software libre (Richard Stallman, Linus Torvalds).
 - **La burbuja .com:** auge y caída, las empresas que sobrevivieron (Amazon, Google, eBay).
- **La era moderna (2010s - actualidad):**
 - **Cloud computing:** AWS, Azure, GCP. Software como servicio (SaaS).

- **Mobile revolution:** iOS, Android, app stores. El smartphone como computadora principal.
- **Open source masivo:** GitHub, npm, NuGet. Comunidades de desarrollo.
- **Inteligencia artificial:** machine learning, ChatGPT, Claude, copilots. IA generativa.
- **DevOps y agilidad:** CI/CD, contenedores (Docker), Kubernetes. Metodologías ágiles.

Actividad del modulo

1. Elaborar una **línea de tiempo ilustrada** con al menos 15 hitos clave de la historia del software, desde Ada Lovelace hasta la era de la IA. Cada hito debe incluir: año, protagonista o tecnología, y una breve descripción de su impacto.
2. Escribir un **ensayo corto (1-2 páginas)** respondiendo: "¿Por qué Ada Lovelace y Alan Turing son considerados los padres de la computación moderna?"

Módulo 3: Tipos de software

Contenido

- **Software de sistema:**
 - Sistemas operativos: Windows, Linux (distribuciones: Ubuntu, Fedora, Arch), macOS, iOS, Android.
 - Drivers: software que traduce entre el SO y el hardware.
 - Firmware: BIOS/UEFI, firmware de routers, firmware de impresoras.
 - Utilidades del sistema: antivirus, desfragmentadores, gestores de archivos.
- **Software de aplicación (desktop):**
 - Productividad: Microsoft Office, LibreOffice, Google Workspace.
 - Diseño: Adobe Photoshop, Figma, AutoCAD.
 - Multimedia: VLC, Spotify, OBS Studio.
 - Videojuegos: desde juegos indie hasta AAA. Motores: Unity, Unreal Engine.
- **Software embebido:**
 - Definición: software que vive dentro de un dispositivo y controla su funcionamiento.
 - Ejemplos: microondas, lavadoras inteligentes, smartwatches, sistemas automotrices, dispositivos médicos.
 - IoT (Internet of Things): dispositivos conectados que recopilan y envían datos.
- **Software móvil:**
 - Apps nativas: desarrolladas para un SO específico (Swift para iOS, Kotlin para Android).
 - Apps híbridas: un solo código para múltiples plataformas (React Native, Flutter, MAUI).
 - Progressive Web Apps (PWA): apps web que se comportan como nativas.
 - App Stores: ecosistema de distribución.
- **Software web:**
 - SPA (Single Page Application): React, Angular, Vue. La página no recarga.
 - MPA (Multi Page Application): cada acción carga una página nueva.
 - PWA (Progressive Web App): funciona offline, se instala como app.
 - Ejemplos: Gmail, redes sociales, e-commerce, plataformas de streaming.
 - Diferencia entre frontend (lo que ve el usuario) y backend (lo que procesa el servidor).
- **Software empresarial:**
 - ERP (Enterprise Resource Planning): SAP, Oracle ERP, Microsoft Dynamics. Gestión integral de la empresa.

- CRM (Customer Relationship Management): Salesforce, HubSpot. Gestión de relaciones con clientes.
- BI (Business Intelligence): Power BI, Tableau. Análisis de datos y reportes.
- Banca digital, sistemas hospitalarios, software logístico.

Actividad del modulo

1. Elaborar un **cuadro comparativo** de los 6 tipos de software con las siguientes columnas: tipo, definición, ejemplos concretos (al menos 3 por tipo), características principales, plataforma donde se ejecuta, y usuario típico.
2. Identificar y listar al menos 10 piezas de software que uses en tu vida diaria, clasificándolas por tipo.

Módulo 4: El ecosistema tecnológico

Contenido

- **IT (Information Technology):** infraestructura tecnológica, redes, servidores, soporte técnico, administración de sistemas. La base sobre la que todo se construye.
- **Fintech (Financial Technology):** pagos digitales (PayPal, Stripe), billeteras (Nequi, Mercado Pago), criptomonedas y blockchain, neobancos (Nubank, N26), plataformas de inversión. Cómo la tecnología está transformando los servicios financieros.
- **Healthtech (Health Technology):** historia clínica electrónica, telemedicina, wearables de salud (smartwatches que miden ritmo cardíaco), software de diagnóstico asistido por IA, gestión hospitalaria.
- **Edtech (Education Technology):** plataformas de e-learning (Coursera, Platzi, Udemy), LMS (Learning Management Systems), realidad virtual/aumentada en educación, gamificación del aprendizaje.
- **IoT (Internet of Things):** sensores, dispositivos conectados, smart homes, smart cities, agricultura de precisión, logística inteligente. Cómo los objetos se conectan a internet y generan datos.
- **GovTech (Government Technology):** gobierno electrónico, trámites digitales, identidad digital, datos abiertos, smart cities. Cómo la tecnología mejora servicios públicos.
- **AgriTech (Agriculture Technology):** drones para monitoreo de cultivos, sensores de suelo, software de gestión agrícola, agricultura de precisión.
- **PropTech (Property Technology):** plataformas de compra/venta/alquiler de inmuebles, tours virtuales, gestión de propiedades.
- **LegalTech (Legal Technology):** automatización de contratos, análisis de documentos legales con IA, plataformas de resolución de disputas online.
- **Cómo cada ecosistema transforma la vida diaria:** impacto social, económico y cultural de la tecnología en cada sector.

Actividad del modulo

1. Elegir **un ecosistema tecnológico** de los presentados y elaborar un **informe investigativo** (2-3 páginas) que incluya:
 - Definición del ecosistema y qué problema resuelve.
 - Al menos 5 empresas reales del sector con una breve descripción de cada una.
 - Impacto en la sociedad (beneficios concretos).
 - Retos y oportunidades del sector.

- Una reflexión personal sobre cómo este ecosistema te afecta o podría afectarte.
2. Elaborar un **diagrama** que muestre cómo se conectan al menos 3 ecosistemas entre sí (por ejemplo: Fintech + Healthtech = seguros de salud digitales).

Módulo 5: Herramientas y roles del desarrollador de software

Contenido

- **Herramientas esenciales del desarrollador:**
 - **IDEs:** Visual Studio 2026, IntelliJ IDEA, Eclipse, Xcode. Qué ofrecen (autocompletado, debugging, refactoring).
 - **Editores de código:** VS Code, Sublime Text, Vim. Más livianos pero potentes con extensiones.
 - **Terminal:** herramienta imprescindible (ya vista en Curso 0).
 - **Navegadores y Dev Tools:** Chrome DevTools, Firefox Developer Tools. Inspección de elementos, consola, red, performance.
 - **Gestores de paquetes:** npm (Node.js), NuGet (.NET), pip (Python). Qué son las dependencias.
 - **Herramientas de diseño/prototipado:** Figma, Adobe XD. Cómo los diseños se convierten en código.
 - **Herramientas de comunicación:** Slack, Microsoft Teams, Discord. Comunicación en equipos de desarrollo.
 - **Gestores de proyectos:** Jira, Azure DevOps, Trello, Linear. Seguimiento de tareas y sprints.
- **Roles en el desarrollo de software:**
 - **Frontend Developer:** construye lo que el usuario ve e interactúa. HTML, CSS, JavaScript, React, Angular.
 - **Backend Developer:** construye la lógica del servidor, APIs, bases de datos. C#, Java, Node.js, Python.
 - **Fullstack Developer:** domina ambos mundos (frontend + backend).
 - **Mobile Developer:** desarrollo de aplicaciones móviles. Swift, Kotlin, React Native, Flutter.
 - **DevOps Engineer:** automatización, CI/CD, infraestructura, contenedores, monitoreo.
 - **QA Engineer (Quality Assurance):** testing manual y automatizado, calidad del software.
 - **DBA (Database Administrator):** diseño, administración y optimización de bases de datos.
 - **Tech Lead / Arquitecto de Software:** decisiones técnicas, diseño de sistemas, guía del equipo.
 - **Scrum Master / Project Manager:** gestión del equipo y del proceso de desarrollo.
 - **UX/UI Designer:** experiencia de usuario, diseño de interfaces, investigación de usuarios.
- **Cómo se organiza un equipo de desarrollo:**
 - Equipos pequeños vs grandes.
 - Metodologías: cascada vs ágil (breve introducción, se profundiza en Curso 15).

- El ciclo de vida de un proyecto: idea → diseño → desarrollo → testing → deployment → mantenimiento.

Actividad del modulo

1. Elaborar una **matriz de roles** con las siguientes columnas: rol, responsabilidades principales, herramientas que usa, lenguajes/tecnologías asociadas, y con qué otros roles interactúa más.
2. Investigar en LinkedIn o en portales de empleo (Computrabajo, Indeed, LinkedIn Jobs) al menos 3 ofertas laborales de diferentes roles de desarrollo de software. Para cada una, anotar: título del puesto, empresa, requisitos técnicos, y salario (si está disponible). Reflexionar sobre cuál le llama más la atención y por qué.

Módulo 6: Tu primer "Hola Mundo"

Contenido

- **¿Qué es un "Hola Mundo"?** Tradición en programación: el primer programa que se escribe en cualquier lenguaje. Por qué existe esta tradición.
- **Crear un proyecto de consola en C# con .NET 10:**
 - Abrir Visual Studio 2026.
 - Crear nuevo proyecto → Aplicación de consola (C#).
 - Seleccionar .NET 10.
 - Entender la estructura del proyecto generado: archivos, carpetas, archivo `.csproj`.
- **Entender el código línea por línea:**

```
// Programa mínimo en C# (.NET 10 - top-level statements)
Console.WriteLine("¡Hola Mundo!");
```

- ¿Qué es `Console`? Una clase del sistema.
- ¿Qué es `WriteLine`? Un método que escribe texto en la consola.
- ¿Qué son las comillas? Delimitan una cadena de texto (string).
- ¿Qué es el punto y coma? Indica fin de instrucción.
- **Ejecutar el programa:** botón de play en VS 2026, o desde terminal con `dotnet run`.
- **Experimentar:** cambiar el mensaje, agregar múltiples líneas, usar `Console.ReadLine()` para leer entrada del usuario.
- **Crear el mismo proyecto desde la terminal:**

```
dotnet new console -n HolaMundo
cd HolaMundo
dotnet run
```

- **Abrir el proyecto en VS Code:** ver las diferencias de experiencia entre ambos IDEs.

Actividad del modulo

1. Crear el programa "Hola Mundo" tanto en Visual Studio 2026 como desde la terminal/VS Code.
2. Modificar el programa para que:
 - Pregunte el nombre del usuario (`Console.ReadLine()`).
 - Salude al usuario por su nombre (`Console.WriteLine($"¡Hola {nombre}!")`).

3. Escribir una **breve reflexión** (1 párrafo) sobre cómo se sintió antes, durante y después de escribir su primer código.
4. Subir todo al repo en la rama correspondiente, hacer commit y PR.

Actividad final del Curso 1

"Panorama del mundo del software"

Desde la rama `curso-1/actividad-final`:

Elaborar un **ensayo investigativo** (3-4 páginas) que integre los conocimientos adquiridos en todos los módulos. El ensayo debe incluir:

1. **Introducción:** breve recorrido por la historia del software, destacando al menos 3 hitos que considere más relevantes y justificando por qué.
2. **Desarrollo:**
 - Elegir un **tipo de software** (ej: web, móvil, embebido) y describir sus características, ventajas y ejemplos.
 - Identificar un **ecosistema tecnológico** donde ese tipo de software opera (ej: Fintech, Healthtech) y describir su impacto.
 - Describir qué **rol de desarrollador** se necesitaría para construir ese software y qué herramientas usaría.
3. **Conclusión:** reflexión personal sobre qué área del desarrollo de software le genera más curiosidad y por qué.
4. **Requisitos formales:**
 - Estructura clara: introducción, desarrollo, conclusión.
 - Al menos 3 fuentes de información citadas.
 - Entrega en formato Markdown (.md) dentro del repositorio.
 - Commit con mensajes siguiendo convención y PR hacia develop.

Criterios de evaluación:

- Profundidad y precisión del contenido.
- Integración de los temas vistos en los 6 módulos.
- Estructura y redacción del ensayo.
- Correcto uso de Markdown y Git (rama, commits, PR).